

Lab 08 – Worksheet

Name: Emaan Fatima, Abdul Ghani, Arsheel Abbas	ID: 11031,11943,11163	Section: T3
--	-----------------------	-------------

Programming Basys-3 board

In this report, include following:

1. AND Gate

- Snapshot of simulation output of AND Gate (Picture of your FPGA Board).

2. Adder-Subtractor

- Add **all** the codes related to adder subtractor (Testbenches not required)
- Design Modules
- Constraint Files
- Picture of your programmed FPGA Board with output displayed on 7-segment display and write the details of output displayed.

Note: Add the data in this worksheet, the pictures must not be blurry. Rubric sheet must be attached at the end of this worksheet.

• Adder-Subtractor:

1) `timescale 1ns / 1ps`

```
module complement (  
    input O,  
    input [2:0] B,  
    output [2:0] C  
);
```

```
// Logic for C0  
assign C[0] = B[0];
```

```
// Logic for C1  
assign C[1] = (~O & B[1]) | (B[1] & ~B[0]) | (O & ~B[1] & B[0]);
```

```
// Logic for C2  
assign C[2] = (~O & B[2]) | (B[2] & ~B[1] & ~B[0]) | (O & ~B[2] & B[0]) | (O & ~B[2] & B[1]);
```

```
endmodule
```

2) `timescale 1ns / 1ps

```
// Main 3-bit Ripple Carry Adder
```

```
module threebitadder (
```

```
    input [2:0] A,
```

```
    input [2:0] B,
```

```
    output [2:0] S,
```

```
    output Cout
```

```
);
```

```
    wire Cout1, Cout2; // Internal wires to connect the carries
```

```
    // Instantiate three 1-bit full adders
```

```
    // The first one has a carry-in of 0
```

```
    Fulladder FA1 (A[0], B[0], 1'b0, S[0], Cout1);
```

```
    Fulladder FA2 (A[1], B[1], Cout1, S[1], Cout2);
```

```
    Fulladder FA3 (A[2], B[2], Cout2, S[2], Cout);
```

```
endmodule
```

```
// Sub-module: 1-bit Full Adder
```

```
module Fulladder (
```

```
    input a,
```

```
    input b,
```

```
    input c_in,
```

```
    output sum,
```

```
    output c_out
```

```
);
```

```
    assign sum = a ^ b ^ c_in; // XOR for sum
```

```
    assign c_out = (a & b) | (a & c_in) | (b & c_in); // AND/OR for carry-out
```

```
endmodule
```

3) `timescale 1ns / 1ps

```
module seven_segment (
```

```
    input [3:0] D,
```

```
    output reg [6:0] S // Changed to 'reg' so we can use it in an always block
```

```
);
```

```
    // S maps to [g, f, e, d, c, b, a]
```

```
    // 0 = ON, 1 = OFF (Active Low)
```

```
    always @(*) begin
```

```
        case(D)
```

```
            4'h0: S = 7'b1000000; // Display 0
```

```
            4'h1: S = 7'b1111001; // Display 1
```

```

4'h2: S = 7'b0100100; // Display 2
4'h3: S = 7'b0110000; // Display 3
4'h4: S = 7'b0011001; // Display 4
4'h5: S = 7'b0010010; // Display 5
4'h6: S = 7'b0000010; // Display 6
4'h7: S = 7'b1111000; // Display 7
4'h8: S = 7'b0000000; // Display 8
4'h9: S = 7'b0010000; // Display 9
4'hA: S = 7'b0001000; // Display A
4'hB: S = 7'b0000011; // Display b
4'hC: S = 7'b1000110; // Display C
4'hD: S = 7'b0100001; // Display d
4'hE: S = 7'b0000110; // Display E
4'hF: S = 7'b0001110; // Display F
default: S = 7'b1111111; // All OFF by default
endcase // <-- This is the fix!
end

```

```
endmodule
```

4) `timescale 1ns / 1ps

```

module ADDERSUB (
    input [2:0] A,
    input [2:0] B,
    input O,
    output [6:0] seg, // Maps to segments a-g on the board
    output dp,       // Decimal point (Minus Sign Display)
    output [3:0] an   // Anode activation for the 4 displays
);

// -----
// 1. Display Activation
// -----
// The Basys 3 has 4 displays. They are active-low, meaning '0' turns it ON.
// We send '0' to the rightmost display and '1' to the rest to keep them OFF[cite: 368, 369,
370, 371, 372].
assign an = 4'b1110;

// -----
// 2. Internal Wires Definition
// -----
wire [2:0] C;      // Output from first complement block
wire [2:0] Sum_U;  // 3-bit sum from the adder
wire Carry_U3;     // Carry-out from the adder

// Wires for the logic gates shown in the block diagram [cite: 135, 145, 148, 149, 150]
wire not1_out;
wire or1_out;
wire not2_out;

```

```

wire and1_out;

wire [2:0] D_lower; // 3-bit output from the second complement block
wire [3:0] D;      // Final 4-bit bus going into the 7-segment decoder

// -----
// 3. Logic Gates Implementation
// -----
assign not1_out = ~O; // NOT_1 [cite: 135]
assign or1_out = not1_out | Carry_U3; // OR_1 [cite: 149]
assign dp = or1_out; // DP is connected to OR_1 output [cite: 149,
151]

assign not2_out = ~or1_out; // NOT_2 (Selector for 2nd complement) [cite:
146, 148]
assign and1_out = not1_out & Carry_U3; // AND_1 (Becomes the MSB for the
decoder) [cite: 150]

// Combine the 1-bit AND output and 3-bit complement output into a 4-bit wire
assign D = {and1_out, D_lower};

// -----
// 4. Sub-Module Instantiations
// -----

// First Selective 2's Complement [cite: 137, 143]
complement comp1 (
    .O(O),
    .B(B),
    .C(C)
);

// 3-Bit Adder [cite: 138, 139, 140, 144, 145]
threebitadder adder (
    .A(A),
    .B(C),
    .S(Sum_U),
    .Cout(Carry_U3)
);

// Second Selective 2's Complement [cite: 140, 141, 146]
complement comp2 (
    .O(not2_out),
    .B(Sum_U),
    .C(D_lower)
);

// 7-Segment Decoder [cite: 141, 142, 147]
seven_segment decoder (
    .D(D),

```

```
.S(seg)
);
```

```
endmodule
```

5) `timescale 1ns / 1ps

```
module ADDERSUBTEST();
```

```
    // Inputs (Registers)
    reg [2:0] A;
    reg [2:0] B;
    reg O;
```

```
    // Outputs (Wires)
    wire [6:0] seg;
    wire dp;
    wire [3:0] an;
```

```
    // Instantiate the Unit Under Test (UUT)
    // Make sure 'ADDERSUB' matches the exact name of your top module
    ADDERSUB uut (
        .A(A),
        .B(B),
        .O(O),
        .seg(seg),
        .dp(dp),
        .an(an)
    );
```

```
    // Apply Test Stimulus
    initial begin
        // Case 1: 3 + 4 [cite: 59]
        A = 3'b011; // 3 in binary
        B = 3'b100; // 4 in binary
        O = 1'b0; // 0 for Addition
        #10; // Wait 10 time units

        // Case 2: 3 - 4 [cite: 60]
        A = 3'b011;
        B = 3'b100;
        O = 1'b1; // 1 for Subtraction
        #10;

        // Case 3: 7 + 7 [cite: 61]
        A = 3'b111; // 7 in binary
        B = 3'b111; // 7 in binary
        O = 1'b0;
        #10;
```

```
// Case 4: 2 - 6 [cite: 62]
A = 3'b010; // 2 in binary
B = 3'b110; // 6 in binary
O = 1'b1;
#10;

// End simulation
$finish;
end
```

```
endmodule
```

6) Constraint Files

```
## Switches
```

```
# A[0] to A[2] -> Switches 0, 1, and 2
```

```
set_property PACKAGE_PIN V17 [get_ports {A[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {A[0]}]
set_property PACKAGE_PIN V16 [get_ports {A[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {A[1]}]
set_property PACKAGE_PIN W16 [get_ports {A[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {A[2]}]
```

```
# B[0] to B[2] -> Switches 3, 4, and 5
```

```
set_property PACKAGE_PIN W17 [get_ports {B[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {B[0]}]
set_property PACKAGE_PIN W15 [get_ports {B[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {B[1]}]
set_property PACKAGE_PIN V15 [get_ports {B[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {B[2]}]
```

```
# Operation Bit 'O' -> Far left switch (Switch 15)
```

```
set_property PACKAGE_PIN R2 [get_ports O]
set_property IOSTANDARD LVCMOS33 [get_ports O]
```

```
## 7 Segment Display
```

```
# Segments a-g (seg[0] is 'a', seg[6] is 'g')
```

```
set_property PACKAGE_PIN W7 [get_ports {seg[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[0]}]
set_property PACKAGE_PIN W6 [get_ports {seg[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[1]}]
set_property PACKAGE_PIN U8 [get_ports {seg[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[2]}]
set_property PACKAGE_PIN V8 [get_ports {seg[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[3]}]
set_property PACKAGE_PIN U5 [get_ports {seg[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[4]}]
set_property PACKAGE_PIN V5 [get_ports {seg[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[5]}]
set_property PACKAGE_PIN U7 [get_ports {seg[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[6]}]
```

```
# Decimal Point (dp)
set_property PACKAGE_PIN V7 [get_ports dp]

set_property IOSTANDARD LVCMOS33 [get_ports dp]

# Anodes to activate the displays (an)
set_property PACKAGE_PIN U2 [get_ports {an[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[0]}]
set_property PACKAGE_PIN U4 [get_ports {an[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[1]}]
set_property PACKAGE_PIN V4 [get_ports {an[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[2]}]
set_property PACKAGE_PIN W4 [get_ports {an[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[3]}]
```

7)



WhatsApp Video 2026-03-17 at 10.01.06 AM.mp4

8) Reflection:

In this lab, we designed and implemented a 3-bit adder-subtractor on the Basys-3 FPGA board. This task helped us understand how arithmetic operations like addition and subtraction can be performed using digital logic circuits. Instead of creating separate circuits, we used a single design with a control input **0** to switch between addition and subtraction, which made the system more efficient and practical.

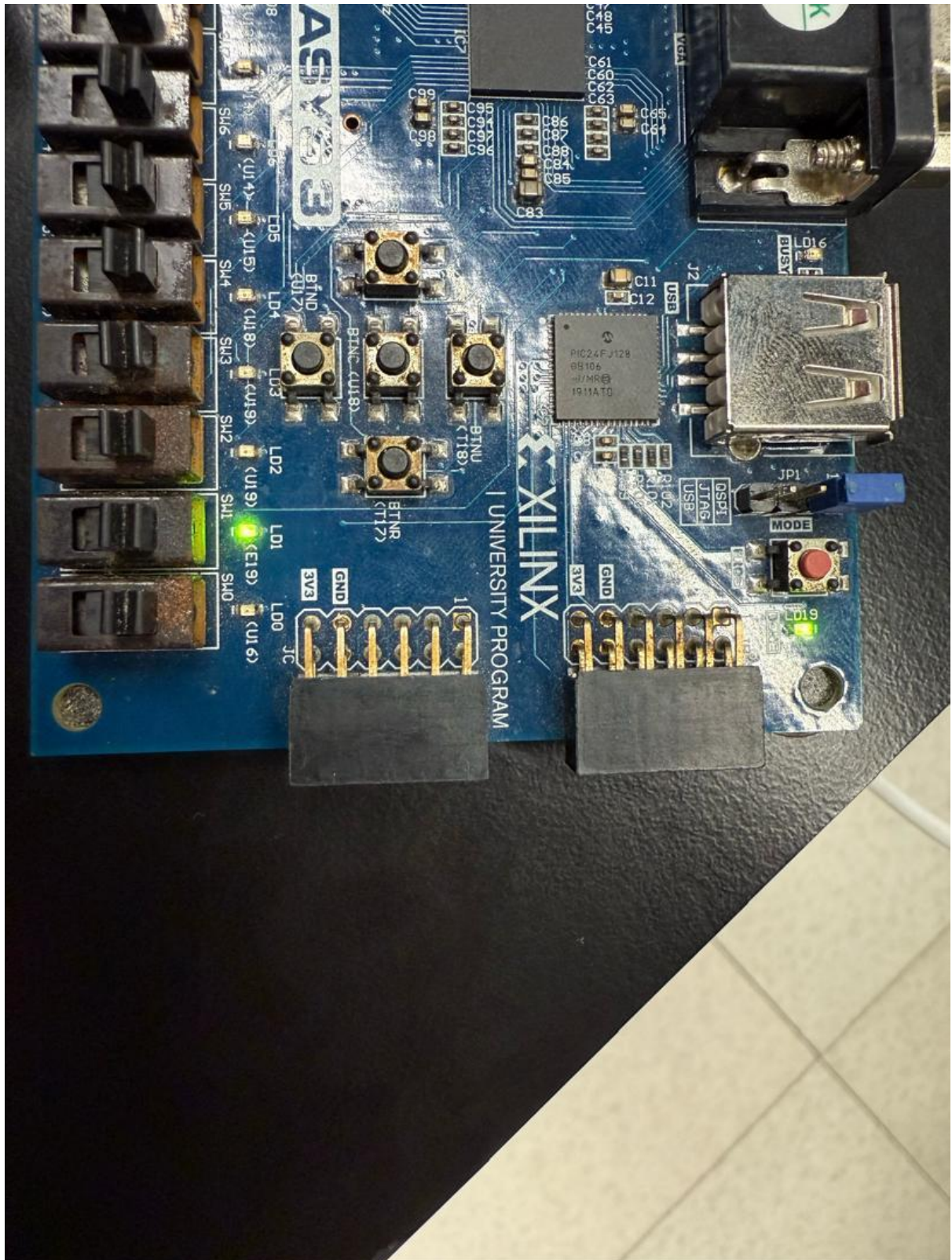
One of the key concepts we learned was the use of **2's complement** for subtraction. By selectively complementing the input B and using a ripple carry adder, we were able to perform subtraction using the same hardware as addition. This showed how powerful and optimized digital design can be.

We also worked with modular design by dividing the system into smaller components such as the complement module, full adder, three-bit adder, and seven-segment display decoder. This made the overall design easier to understand, debug, and implement. Connecting these modules together gave us hands-on experience with hierarchical design in Verilog.

Another important part was displaying the output on the **7-segment display** of the FPGA board. This helped us visually verify the results and understand how binary outputs are converted into human-readable form. The use of the decimal point (dp) to indicate negative results was especially interesting and improved the usability of the design.

Overall, this lab improved our understanding of digital circuits, FPGA programming, and Verilog coding. It also strengthened our problem-solving skills and gave us practical experience in implementing real hardware-based systems.

- **AND GATE:**



Assessment Rubrics

Marks Distribution:

		LR2 Code/Simulation (In-Lab)	LR5 Results	LR7 Viva	LR9 Report
In-lab	Task b	-	10 points	10 points	10 points
	Task c	-	10 points		30 points
	Task d	20 points	10 points		
Total marks 100		20	30	10	40

Marks Obtained:

		LR2 Code/Simulation (In-Lab)	LR5 Results	LR7 Viva	LR9 Report
In-lab	Task b	-			
	Task c	-			
	Task d				
Total marks 100					